COMP 3311
DATABASE MANAGEMENT SYSTEMS

LECTURE 23 EXERCISES
TRANSACTION MANAGEMENT: CONCURRENCY CONTROL PROTOCOLS

EXERCISE 1

Show that the following schedule is conflict serializable according to 2PL by adding lock-s(), lock-x() and unlock() instructions, as necessary, to the schedule. If possible, add the lock-s(), lock-x() and unlock() instructions so that *no transaction is required to wait*.

T_1	T_2	T_3
read(X)		
	read(X)	
		read(Y)
read(Z)		
	read(Y)	
	write(X)	
		read(X)
		write(X)
write(Z)		

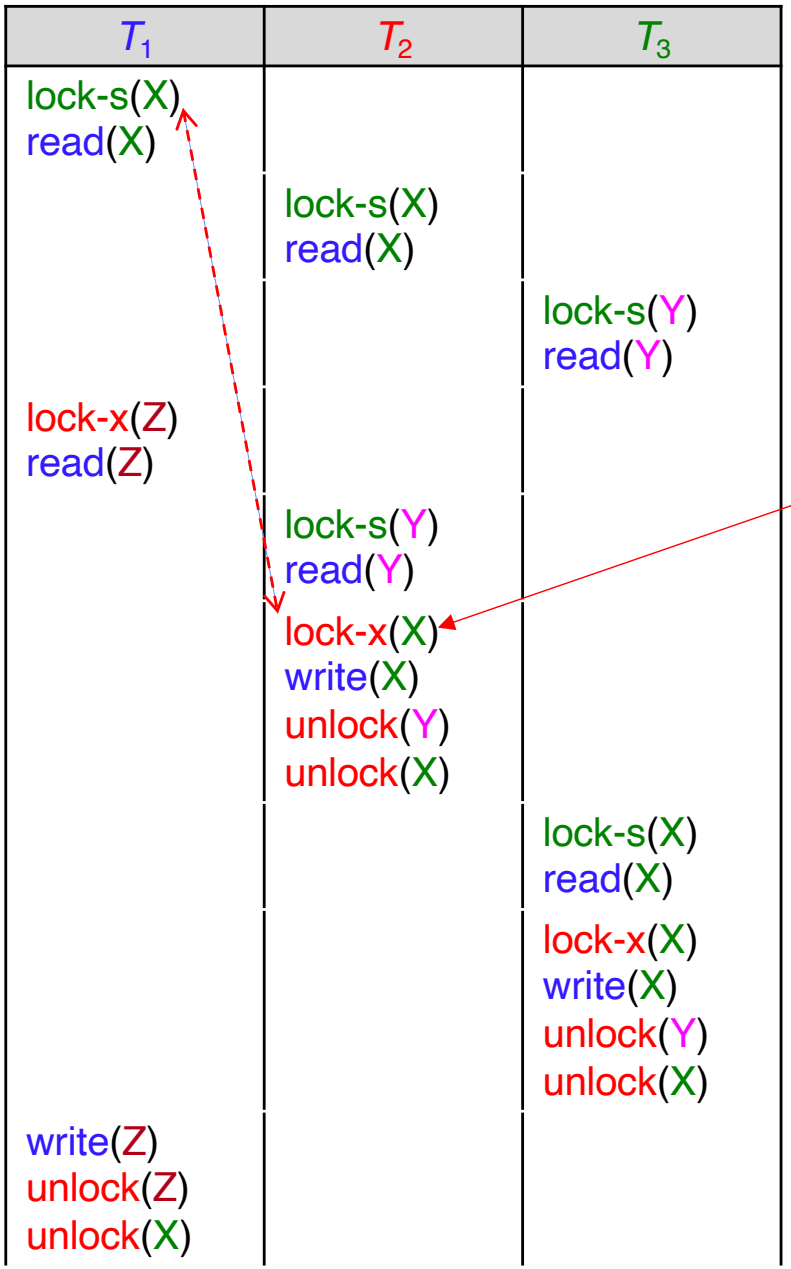
EXERCISE 1 (CONT'D)

T_1	T_2	T_3
lock-s(X)		
read(X)		
unlock(X)		
	lock-s(X)	
	read(X)	
		lock-s(Y)
		read(Y)
lock-x(Z)		
read(Z)		
	lock-s(Y)	
	read(Y)	
	lock-x(X)	
	write(X)	
	unlock(Y)	
	unlock(X)	
		lock-s(X)
		read(X)
		lock-x(X)
		write(X)
		unlock(Y)
		unlock(X)
write(Z)		
unlock(Z)		

Do you see any problem with this schedule?

Under 2PL, a transaction cannot request locks after it has released any lock.

EXERCISE 1 (CONT'D)



Do you see any problem with this schedule?

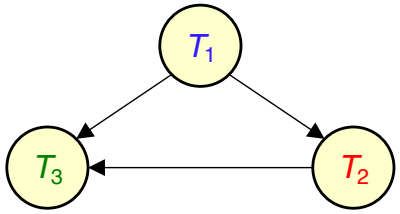
This lock cannot be granted; T_2 must wait.
How to add the locks so that T_2 does not have to wait?

EXERCISE 1 (CONT'D)

T_1	T_2	T_3
lock-s(X) ✓		
read(X)		
	lock-s(X) ✓	
	read(X)	
		lock-s(Y) ✓
		read(Y)
lock-x(Z) ✓		
unlock(X) ✓		
read(Z)		
	lock-s(Y) ✓	
	read(Y)	
	lock-x(X) ✓	
	write(X)	
	unlock(Y) ✓	
	unlock(X) ✓	
		lock-s(X) ✓
		read(X)
		lock-x(X) √
		write(X)
		unlock(Y) ✓
		unlock(X) ✓
write(Z)		
unlock(Z) ✓		

Recall: Under 2PL, a transaction cannot request locks after it has released any lock.

Precedence graph



The schedule is conflict serializable.

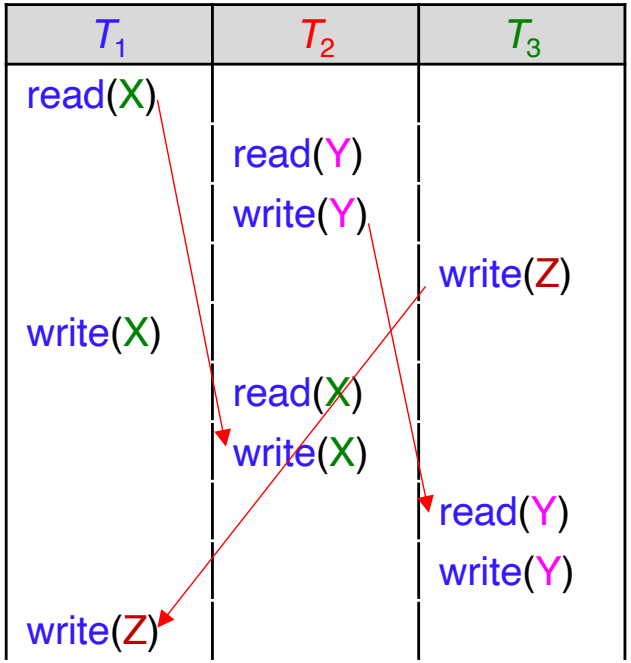
The equivalent serial schedule is $T_1 T_2 T_3$.

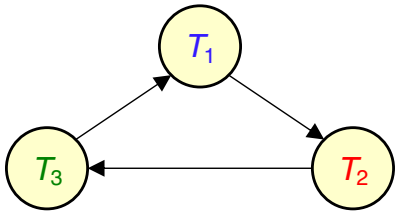
EXERCISE 2

Consider the schedule shown below.

a) Is the schedule conflict serializable?

If yes, give the equivalent serial schedule.





Precedence Graph

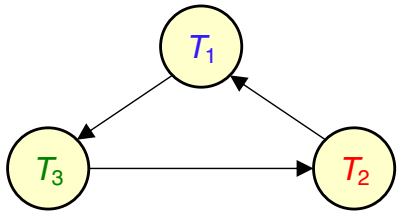
The schedule is not conflict serializable because there is a cycle in the precedence graph.

Therefore, the schedule will fail under any protocol that aims at conflict serializability.

EXERCISE 2 (CONTD)

T_1	T_2	T_3
lock-s(X) ✓		
read(X)		
	lock-s(Y) ✓	
	read(Y)	
	lock-x(Y) ✓	
	write(Y)	
		lock-x(Z) ✓
		write(Z)
lock-x(X) ✓		
write(X)		
	lock-s(X) ✗ cannot be granted – T_2 must wait	
	read(X)	
	write(X)	
		lock-s(Y) ✗ cannot be granted – T_3 must wait
		read(Y)
		write(Y)
lock-x(Z) ✗ cannot be granted – T_1 must wait		
write(Z)	deadlock!	

b) Modify the schedule according to strict 2PL by adding a lock-s() before reading a data item, a lock-x() before writing a data item and an unlock() after all read/write instructions have completed. Is the schedule deadlock free?



Wait-for graph

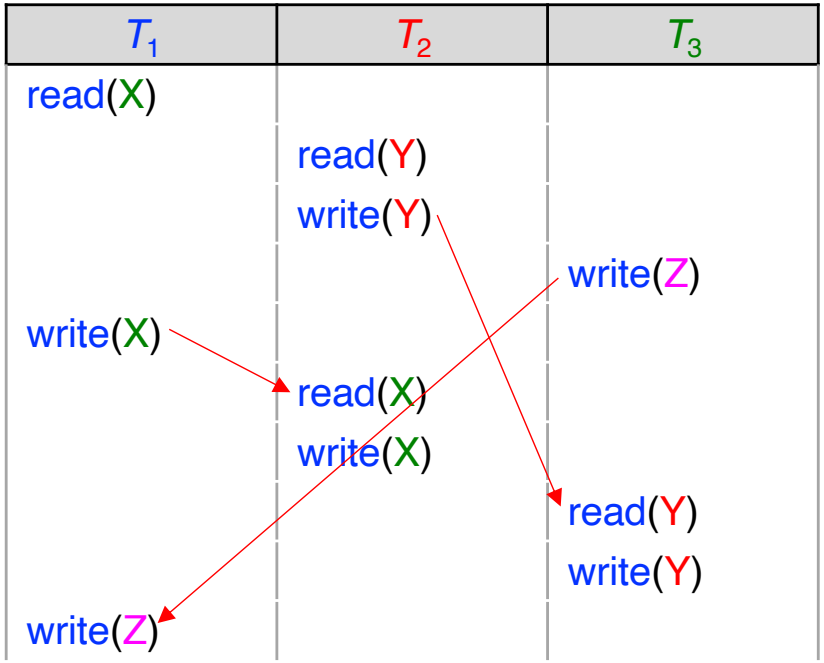
Strict 2PL

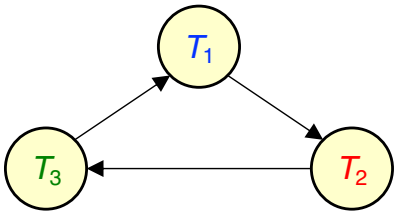
All exclusive-mode locks must be held until a transaction commits (shared-mode locks can be released anytime).

Cannot request any lock after releasing a lock.

EXERCISE 3

Recall that the schedule of Exercise 2 is not serializable because there is a cycle T_1 T_2 T_3 T_1 in the precedence graph. Therefore, the schedule should fail under any protocol that aims at conflict serializability.





Precedence Graph

EXERCISE 3 (CONTD)

☞ The timestamps determine the serializability order.

Use the timestamp-ordering protocol to complete the following non-serializable schedule assuming the timestamps 1, 2, and 3 for transactions T_1, T_2, and T_3, respectively. Show where the protocol will fail. Assume that the initial R/W timestamps of all data items is 0.

Read
If TS(T_i) < WTS(Q) **rollback**
If TS(T_i) ≥ WTS(Q)
RTS(Q) = max(TS(T_i), RTS(Q))

Write
If TS(T_i) < RTS(Q) **rollback**
If TS(T_i) < WTS(Q) **rollback**
Otherwise WTS(Q) = TS(T_i)

RTS(X)=2 | WTS(X)=2 | RTS(Y)=3 | WTS(Y)=3 | RTS(Z)=0 | WTS(Z)=3

T_1 [TS=1]	T_2 [TS=2]	T_3 [TS=3]
read(X) ✓ TS(T_1)=1 ≥ WTS(X)=0; **set** RTS(X)=1		
	read(Y) ✓ TS(T_2)=2 ≥ WTS(Y)=0; **set** RTS(Y)=2	
	write(Y) ✓ TS(T_2)=2 ≥ RTS(Y)=2 & ≥ WTS(Y)=0; **set** WTS(Y)=2	
		write(Z) ✓ TS(T_3)=3 ≥ RTS(Z)=0 & ≥ WTS(Z)=0; **set** WTS(Z)=3
write(X) ✓ TS(T_1)=1 ≥ RTS(X)=1 & ≥ WTS(X)=0; **set** WTS(X)=1		
	read(X) ✓ TS(T_2)=2 ≥ WTS(X)=1; **set** RTS(X)=2	
	write(X) ✓ TS(T_2)=2 ≥ RTS(X)=2 & ≥ WTS(X)=1; **set** WTS(X)=2	
		read(Y) ✓ TS(T_3)=3 ≥ WTS(Y)=2; **set** RTS(Y)=3
		write(Y) ✓ TS(T_3)=3 ≥ RTS(Y)=3 & ≥ WTS(Y)=2; **set** WTS(Y)=3
write(Z) TS(T_1)=1 < WTS(Z)=3 ⟹ rollback (since TS(T_1) < WTS(Z))		

EXERCISE 3 (CONTD)

☞ The timestamps determine the serializability order.

Use the timestamp-ordering protocol to complete the following non-serializable schedule assuming the timestamps 1, 2, and 3 for transactions T_1, T_2, and T_3, respectively. Show where the protocol will fail. Assume that the initial R/W timestamps of all data items is 0.

Read
If TS(T_i) < WTS(Q) **rollback**
If TS(T_i) ≥ WTS(Q)
RTS(Q) = max(TS(T_i), RTS(Q))

Write
If TS(T_i) < RTS(Q) **rollback**
If TS(T_i) < WTS(Q) **ignore**
Otherwise WTS(Q) = TS(T_i)

RTS(X)=2 | WTS(X)=2 | RTS(Y)=3 | WTS(Y)=3 | RTS(Z)=0 | WTS(Z)=3

T_1 [TS=1]	T_2 [TS=2]	T_3 [TS=3]
read(X) ✓ TS(T_1)=1 ≥ WTS(X)=0; **set** RTS(X)=1		
	read(Y) ✓ TS(T_2)=2 ≥ WTS(Y)=0; **set** RTS(Y)=2	
	write(Y) ✓ TS(T_2)=2 ≥ RTS(Y)=2 & ≥ WTS(Y)=0; **set** WTS(Y)=2	
		write(Z) ✓ TS(T_3)=3 ≥ RTS(Z)=0 & ≥ WTS(Z)=0; **set** WTS(Z)=3
write(X) ✓ TS(T_1)=1 ≥ RTS(X)=1 & ≥ WTS(X)=0; **set** WTS(X)=1		
	read(X) ✓ TS(T_2)=2 ≥ WTS(X)=1; **set** RTS(X)=2	
	write(X) ✓ TS(T_2)=2 ≥ RTS(X)=2 & ≥ WTS(X)=1; **set** WTS(X)=2	
		read(Y) ✓ TS(T_3)=3 ≥ WTS(Y)=2; **set** RTS(Y)=3
		write(Y) ✓ TS(T_3)=3 ≥ RTS(Y)=3 & ≥ WTS(Y)=2; **set** WTS(Y)=3
write(Z) TS(T_1)=1 < WTS(Z)=3 ⟹ ignore (since TS(T_1) < WTS(Z))		

Q_k is the version of Q whose write timestamp is the largest write timestamp *less than or equal to* TS(T_i).

EXERCISE 4

☞ The timestamps are used to label versions.

Complete the schedule of Exercise 3 using the multi-version, timestamp-ordering protocol. Assume timestamps 1, 2, and 3 for transactions T_1, T_2, and T_3, respectively, and that the initial R/W timestamps of X_0, Y_0 and Z_0 is 0.

Read
Reads always succeed
set RTS(Q_k) =
max(TS(T_i), RTS(Q_k))

Write
If TS(T_i) < RTS(Q_k) **rollback**
If TS(T_i) = WTS(Q_k)
overwrite contents
If TS(T_i) > WTS(Q_k)
create new version
set R/WTS(Q')=TS(T_i)

RTS(X_0)=1	WTS(X_0)=0	X_0=2
RTS(X_1)=2	WTS(X_1)=1	X_1=3
RTS(X_2)=2	WTS(X_2)=2	X_2=4

RTS(Y_0)=2	WTS(Y_0)=0	Y_0=1
RTS(Y_1)=3	WTS(Y_1)=2	Y_1=5
RTS(Y_2)=3	WTS(Y_2)=3	Y_2=7

RTS(Z_0)=0	WTS(Z_0)=0	Z_0=6
RTS(Z_1)=3	WTS(Z_1)=3	Z_1=8

T_1 [TS=1]	T_2 [TS=2]	T_3 [TS=3]
read(X_0) ✓ set RTS(X_0)=1		
	read(Y_0) ✓ set RTS(Y_0)=2	
	write(Y_1) ✓ TS(T_2)=2 > WTS(Y_0)=0; create Y_1=5; set R/WTS(Y_1)=2	
		write(Z_1) ✓ TS(T_3)=3 > WTS(Z_0)=0; create Z_1=8; set R/WTS(Z_1)=3
write(X_1) ✓ TS(T_1)=1 > WTS(X_0)=0; create X_1=3; set R/WTS(X_1)=1		
	read(X_1) ✓ set RTS(X_1)=2	
	write(X_2) ✓ TS(T_2)=2 > WTS(X_1)=1; create X_2=4; set R/WTS(X_2)=2	
		read(Y_1) ✓ set RTS(Y_1)=3
		write(Y_2) ✓ TS(T_3)=3 > WTS(Y_1)=2; create Y_2=7; set R/WTS(Y_2)=3
write(Z) TS(T_1)=1 > WTS(Z_0)=0; create Z_2=9; set R/WTS(Z_2)=1	⟹ No transaction will ever read this value so it can be deleted!	

Q_k is the version of Q whose write timestamp is the largest write timestamp *less than or equal to* TS(T_i).

EXERCISE 4 (CONT'D)

☞ The timestamps are used to label versions.

The multi-version timestamp-ordering protocol schedule assuming the timestamps 2, 1 and 3 for transactions T_1, T_2 and T_3, respectively.

RTS(X_0)=2	WTS(X_0)=0	X_0=2
RTS(X_1)=2	WTS(X_1)=2	X_1=3

RTS(Y_0)=1	WTS(Y_0)=0	Y_0=1
RTS(Y_1)=1	WTS(Y_1)=1	Y_1=5

RTS(Z_0)=0	WTS(Z_0)=0	Z_0=6
RTS(Z_1)=3	WTS(Z_1)=3	Z_1=8

T_1 [TS=2]	T_2 [TS=1]	T_3 [TS=3]
read(X_0) ✓ set RTS(X_0)=2		
	read(Y_0) ✓ set RTS(Y_0)=1	
	write(Y_1) ✓ TS(T_2)=1 > WTS(Y_0)=0; create Y_1=5; set R/WTS(Y_1)=1	
		write(Z_1) ✓ TS(T_3)=3 > WTS(Z_0)=0; create Z_1=8; set R/WTS(Z_1)=3
write(X_1) ✓ TS(T_1)=2 > WTS(X_0)=0; create X_1=3; set R/WTS(X_1)=2		
	read(X_1) ✓ RTS(X_1)=2	
	write(X) TS(T_2)=1 < RTS(X_0)=2 ⟹ rollback (since TS(T_2) < RTS(X_0))	
		read(Y)
		write(Y)
write(Z)		

Read

Reads always succeed

set RTS(Q_k) = max(TS(T_i), RTS(Q_k))

Write

If TS(T_i) < RTS(Q_k) **rollback**

If TS(T_i) = WTS(Q_k)
overwrite contents

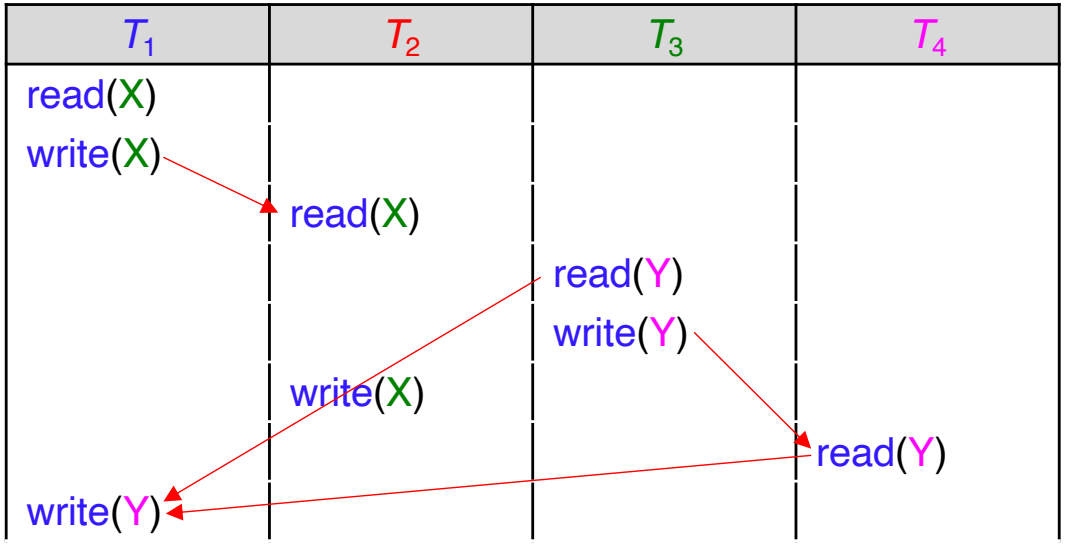
If TS(T_i) > WTS(Q_k)
create new version
set R/WTS(Q')=TS(T_i)

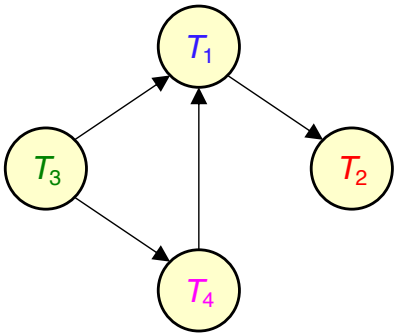
Any other timestamp ordering of the transactions, as in this example, will lead to a rollback.

EXERCISE 5

Consider the schedule shown below.

a) Is the schedule conflict serializable? If yes, give the equivalent serial schedule.





Precedence graph

The schedule is conflict serializable. The equivalent serial schedule is $T_3\,T_4\,T_1\,T_2$.

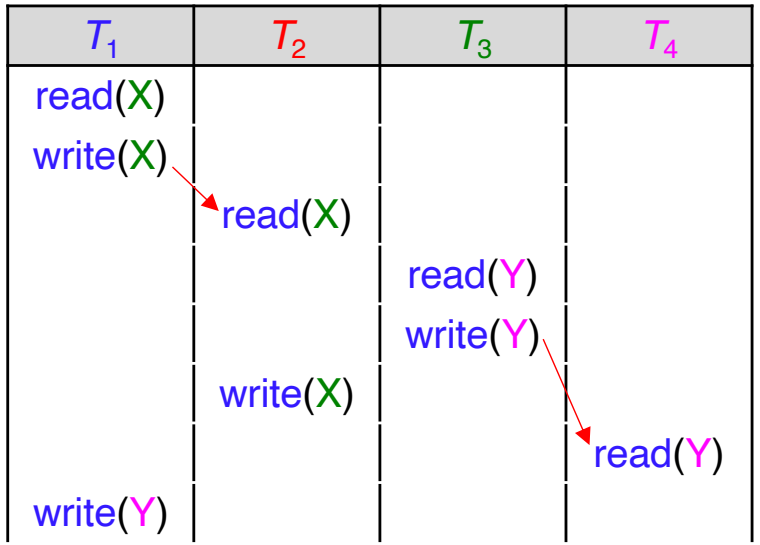
EXERCISE 5 (CONT'D)

b) If T_3 aborts at the end of this schedule, which other transactions will be rolled back?

T_4 because after write(Y) in T_3, the Y read by T_4 is corrupted. Note that the write(Y) of T_1 is not affected by T_3 as it is a blind write (i.e., no read before write) and in the serialization order $T_3 T_4 T_1 T_2$ the write(Y) of T_1 would come after the write(Y) of T_3 and overwrite it.

c) If T_1 aborts at the end of this schedule, which other transactions will be rolled back?

T_2 because after write(X) in T_1, the X read by T_2 is corrupted.

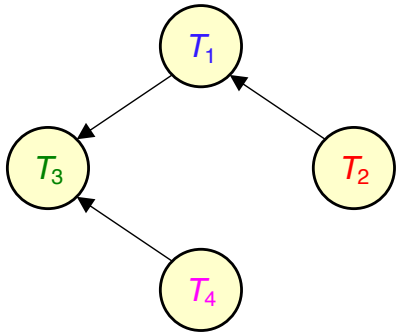


The schedule is conflict serializable. The equivalent serial schedule is $T_3 T_4 T_1 T_2$.

EXERCISE 5 (CONT'D)

d) Construct the wait-for graph that results from this schedule if all locks are only exclusive-locks (lock-x), no locks are released, and the execution process runs to the point of lock-x(Y) just before write(Y) in T_1.

T_1	T_2	T_3	T_4
lock-x(X) ✓			
read(X)			
write(X)			
	lock-x(X) ✗ cannot be granted – T_2 must wait for T_1		
	read(X)		
		lock-x(Y) ✓	
		read(Y)	
		write(Y)	
	write(X)		
			lock-x(Y) ✗ cannot be granted – T_4 must wait for T_3
			read(Y)
lock-x(Y) ✗ cannot be granted – T_1 must wait for T_3			
write(Y)			



Wait-for Graph

Is the system in a deadlock state?

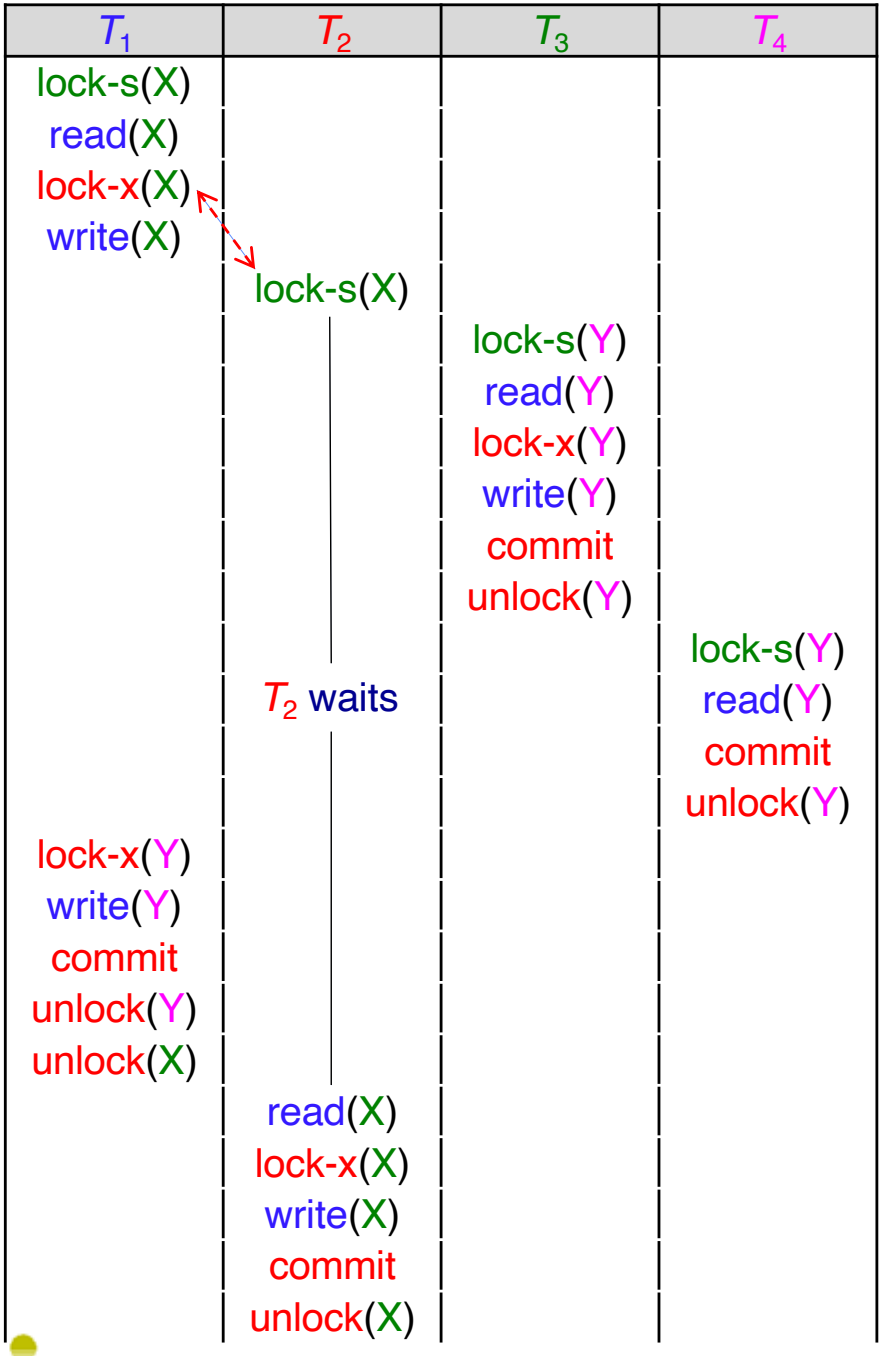
No

EXERCISE 5 (CONTD)

e) Add lock-s(), lock-x() and unlock() instructions to the schedule according to strict 2PL. Indicate which transactions, if any, are required to wait and which instructions cause the wait.

Recall: Under strict 2PL, a transaction must hold all its exclusive-mode locks until it commits.

Normally, a transaction will request a lock-s before it reads a data item and upgrade to a lock-x later if it writes the data item.

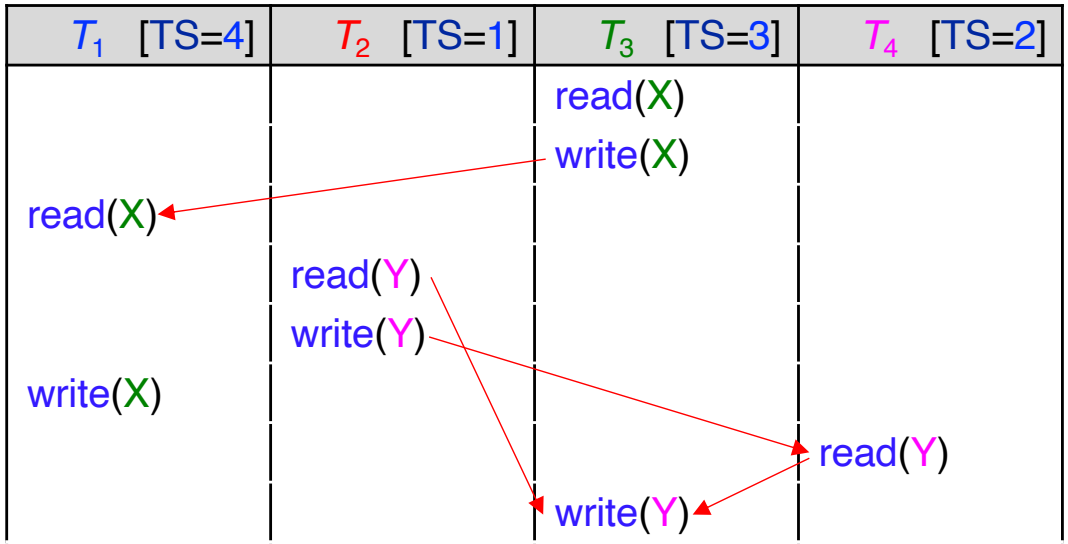


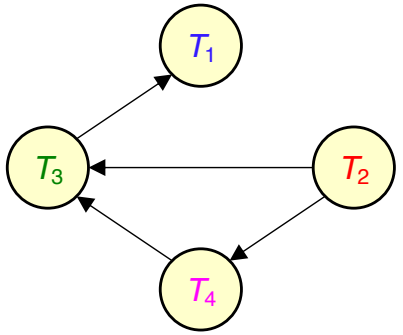
EXERCISE 6

a) The following schedule is conflict serializable.

i. What is the equivalent serial schedule?

☞ The timestamps determine the serializability order.





Precedence graph

The equivalent serial schedule is T_2 T_4 T_3 T_1

ii. Assign appropriate timestamps to the transactions T_1, T_2, T_3 and T_4 so that the schedule is conflict serializable according to the single version, timestamp-ordering protocol. Assume initial R/W timestamp of all data items is 0.

EXERCISE 6 (CONTD)

☞ The timestamps determine the serializability order.

Note that for this schedule any other order of timestamps will fail according to the single version timestamp-ordering protocol as shown below for the order T_1, T_2, T_3, T_4.

Read
If TS(T_i) < WTS(Q) **rollback**
If TS(T_i) ≥ WTS(Q)
RTS(Q) = max(TS(T_i), RTS(Q))

Write
If TS(T_i) < RTS(Q) **rollback**
If TS(T_i) < WTS(Q) **ignore**
Otherwise WTS(Q) = TS(T_i)

RTS(X)=3 | WTS(X)=3 | RTS(Y)=0 | WTS(Y)=0

T_1 [TS=1]	T_2 [TS=2]	T_3 [TS=3]	T_4 [TS=4]
		read(X) ✓ TS(T_3)=3 ≥ WTS(X)=0; set RTS(X)=3	
		write(X) ✓ TS(T_3)=3 ≥ RTS(X)=3 & ≥ WTS(X)=0; **set** WTS(X)=3	
read(X) TS(T_1)=1 < WTS(X)=3	⟹ rollback (since TS(T_1) < WTS(X))		
	read(Y)		
	write(Y)		
write(X)			
			read(Y)
		write(Y)	

Q_k is the version of Q whose write timestamp is the largest write timestamp *less than or equal to* TS(T_i).

EXERCISE 6 (CONTD)

☞ The timestamps determine the serializability order.

b) Use the multi version, timestamp-ordering protocol, to complete the conflict serializable schedule of a) assuming the timestamps 1, 2, 3, and 4 for transactions T_1, T_2, T_3 and T_4, respectively. Show where the schedule will fail. Assume that the initial R/W timestamp of all data items is 0.

Read
Reads always succeed
set RTS(Q_k) = max(TS(T_i), RTS(Q_k))

Write
If TS(T_i) < RTS(Q_k) **rollback**
If TS(T_i) = WTS(Q_k) **overwrite contents**
If TS(T_i) > WTS(Q_k) **create new version**
set R/WTS(Q')=TS(T_i)

RTS(X_0)=3	WTS(X_0)=0	X_0=2	RTS(Y_0)=2	WTS(Y_0)=0	Y_0=1
RTS(X_1)=3	WTS(X_1)=3	X_1=5	RTS(Y_1)=4	WTS(Y_1)=2	Y_1=7

T_1 [TS=1]	T_2 [TS=2]	T_3 [TS=3]	T_4 [TS=4]
		read(X_0) ✓ read X_0=2; set RTS(X_0)=3	
		write(X_1) ✓ TS(T_3)=3 ≥ RTS(X_0)=3 & > WTS(X_0)=0; create X_1=5; set R/WTS(X_1)=3	
read(X_0) ✓ read X_0=2			
	read(Y_0) ✓ read Y_0=1; set RTS(Y_0)=2		
	write(Y_1) ✓ TS(T_2)=2 ≥ RTS(Y_0)=2 & > WTS(Y_0)=0; create Y_1=7; set R/WTS(Y_1)=2		
write(X) TS(T_1)=1 < RTS(X_0)=3 ⟹ rollback (since TS(T_1) < RTS(X_0))			
			read(Y_1) ✓ read Y_1=7; set RTS(Y_1)=4
		write(Y) TS(T_3)=3 < RTS(Y_1)=4 ⟹ rollback (since TS(T_3) < RTS(Y_1))	